

Modelos e arquiteturas de armazenamento de grandes volumes de dados e Bancos de dados NoSQL

Prof. Sergio Serra
serra@ppgi.ufrj.br

BD II – 2026/1

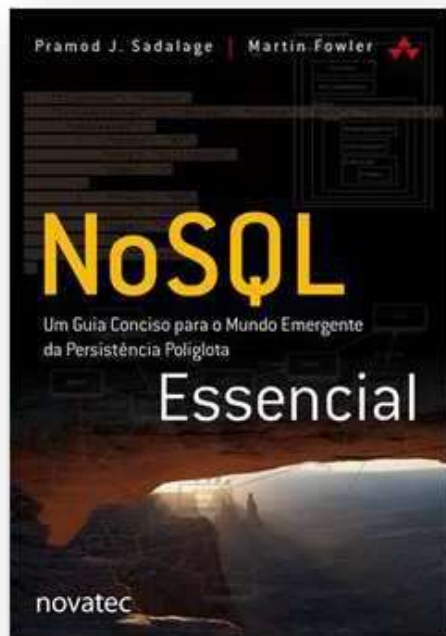
AULA 1

Objetivos da aula

1. Apresentar os principais conceitos de Bancos de Dados NoSQL.
2. Discutir os principais aspectos dos modelos e arquiteturas de Big Data e NoSQL na ótica da Engenharia de Dados.
3. Apresentar exemplos e casos de uso no cenário brasileiro

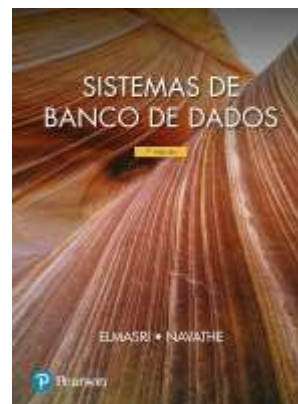
Referências Bibliográficas da Aula

Livro Texto

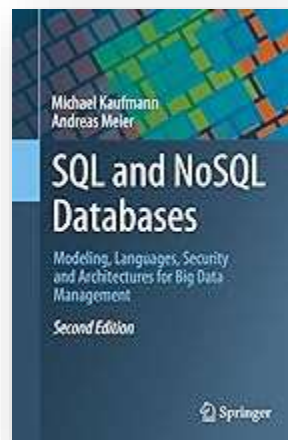


Sadalage, P. E Fowler, M
NoSQL Essencial: um Guia Conciso Para o
Mundo Emergente da Persistência Poliglota
Ed. Novatec , 2013.
ISBN 978-8575223383

Complementar



Elmasri, R., Navathe S., B.
Sistema de Banco de Dados
Ed. Pearson, 2019.
ISDB 978-8543025001



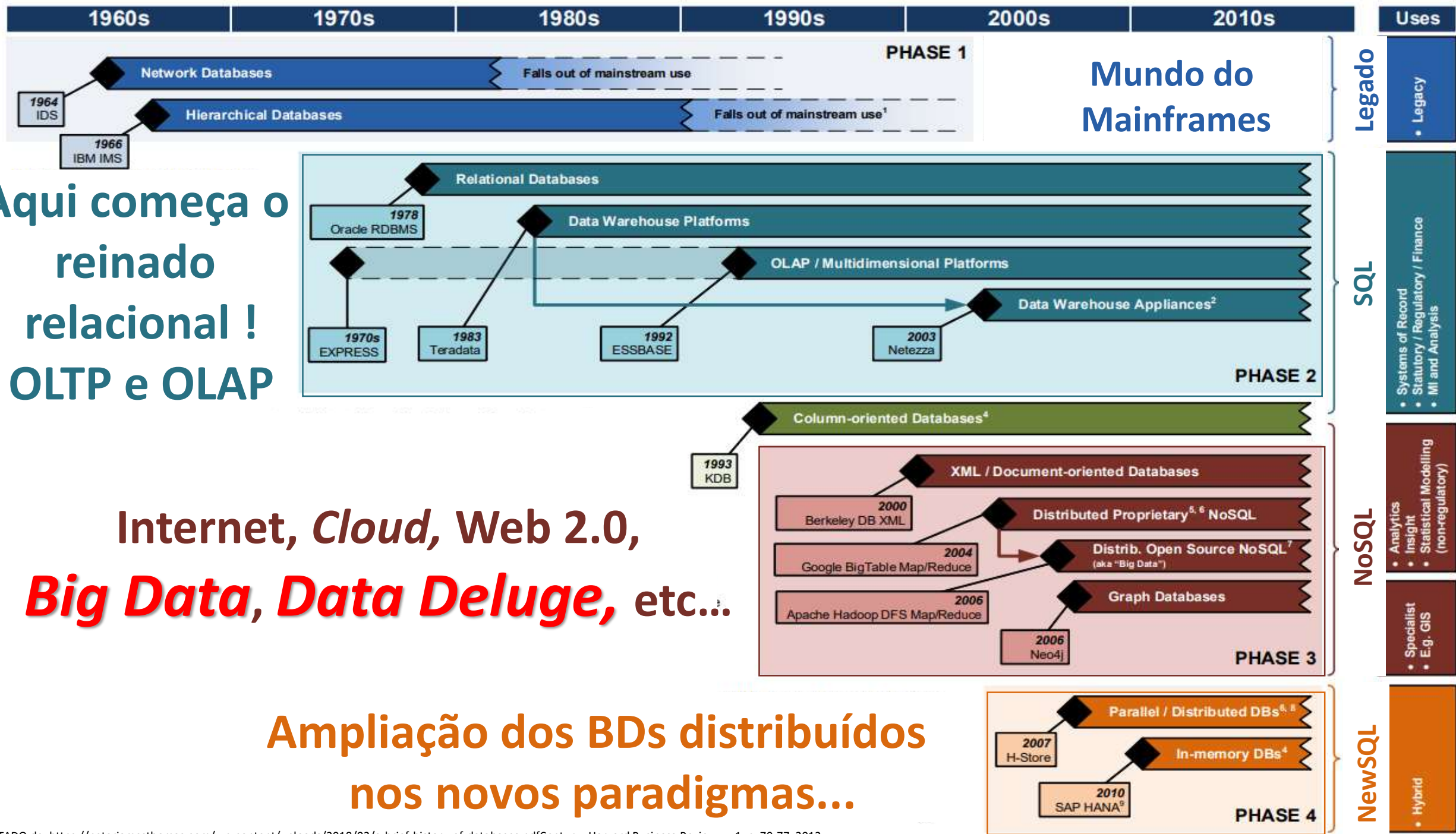
Kaufmann, M. e Meier, A.
SQL and Nosql Databases: Modeling,
Languages, Security and Architectures for Big
Data Management
Ed. Springer, 2023.
ISBN 978-3031279072

Conteúdos da aula

- Big Data
- Motivação para NoSQL
- O reinado dos SGBDr está ameaçado?
- Conceitos básicos para o NoSQL
 - Agregados, Fragmentação e Replicação de Dados
 - Modelos de dados em NoSQL
 - Teorema CAP
- Visão geral dos 4 tipos de sistemas NoSQL e exemplos
- Exercícios e Leituras Complementares

Motivação

- **SBGDs evoluem para acompanhar os novos cenários de computação e seus novos desafios (Big Data, IOT, xIA, Medias Sociais, Wearables, Web de Dados, etc..)**
- **Máquinas evoluem..**
 - **Tipos de memória RAM :** 256KB (1980) → Terabytes de DDR5
 - **Tipos de disco:** HD → SSD
 - **Tipos de Processadores:** 386 → Pentium → Core I9
- **Ambientes evoluem...**
 - **Arquitetura de Sistemas:** Centralizado → Distribuído (C/S, P2P, Clusters, HPC, Cloud)
 - **Tipos de processadores :** CPU Z80 → Intel I9 → GPU Nvidia
 - **Workload:** OLTP → OLAP → HTAP
 - **Redes E celulares :** LAN → Internet → 5G



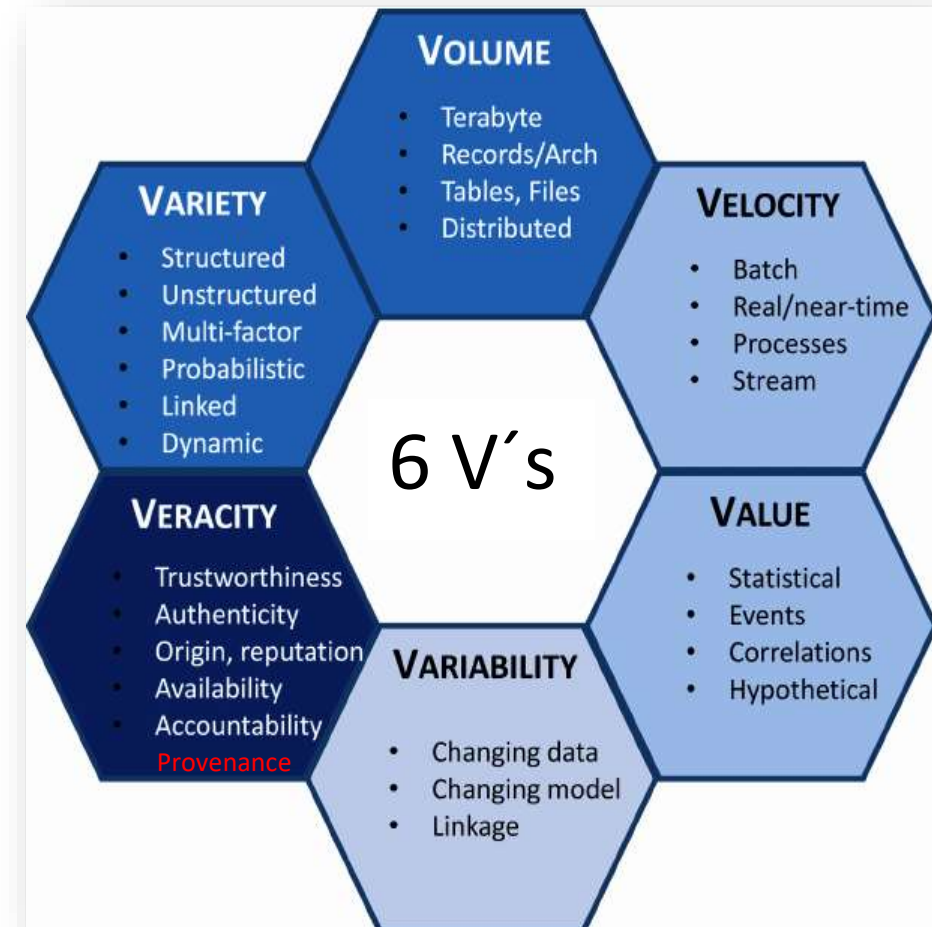
Aqui começa o reinado relacional ! OLTP e OLAP

Internet, Cloud, Web 2.0, **Big Data, Data Deluge**, etc...

Ampliação dos BDs distribuídos nos novos paradigmas...

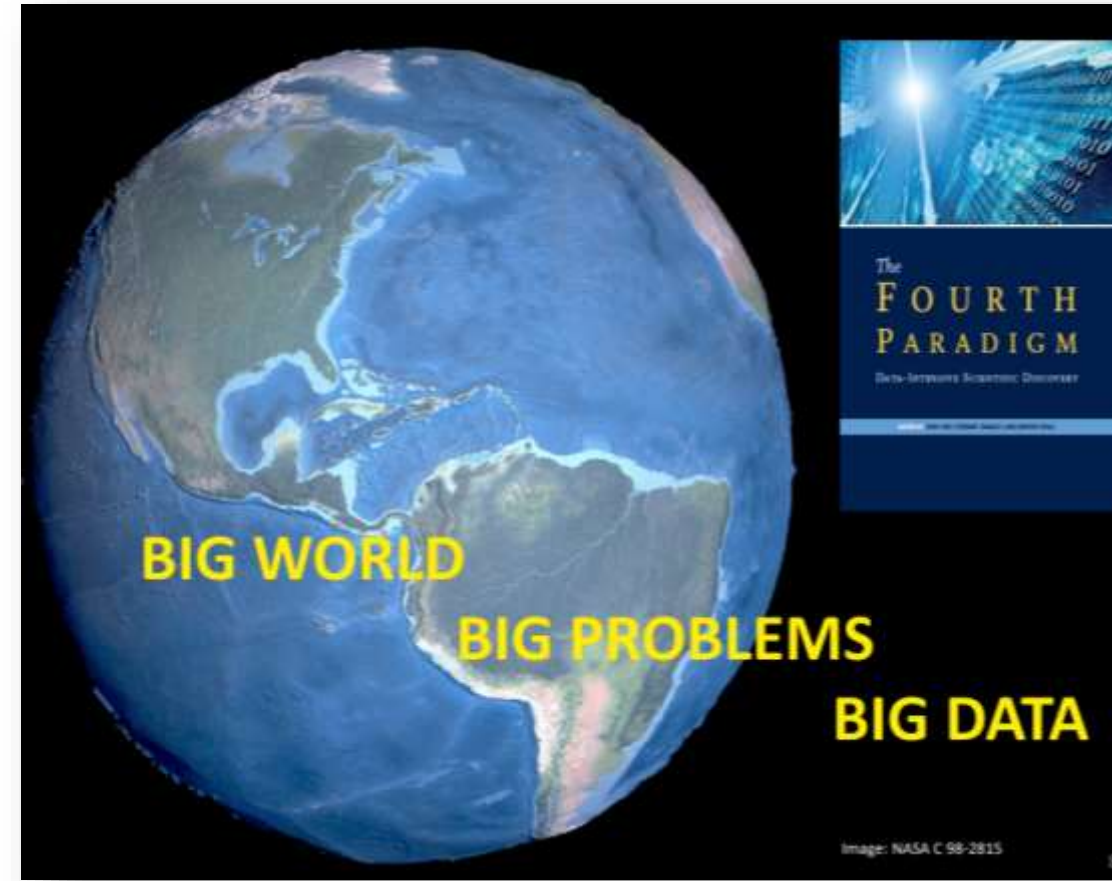
Big Data

- Big data é um grande volume de dados, alta velocidade e alta variedade de **ativos de informação**, demandam formas **inovadoras e econômicas de processamento** de dados para melhor insight e tomada de decisões.
- **Aplicações computacionais de Ciência dos Dados no Big Data** tem o objetivo de analisar e extrair valor de grandes volumes de dados usando **novas técnicas computacionais**.
- Os desafios para **gestão dos dados** são **classificados em termos de V's (3, 4, 5, 6 ...)** são **amplos** (Chen et al., 2012, Kwon et al., 2014).



Data Deluge

- Paradigmas das Ciências...
 - Há centenas de anos a ciência era empírica e observacional
 - Há poucas décadas passou também a ser teórica e usar modelos
 - Nas últimas décadas passou a ser validar os modelos em simulações no computador
- **Hoje é orientada a grandes volumes de dados (*in silico*)**
 - Instrumentação sofisticada
 - Computação distribuída
 - Novos softwares, etc...
- **Big Data (e seus V's) e Data Science e IA não são mais exclusividade da CC!**



Armazenamento de Grande Volumes de Dados

1 linha de 80 caracteres



5MB/disco



1MB/fita



30MB/disco



HDD



RAID



SSD



Data Center



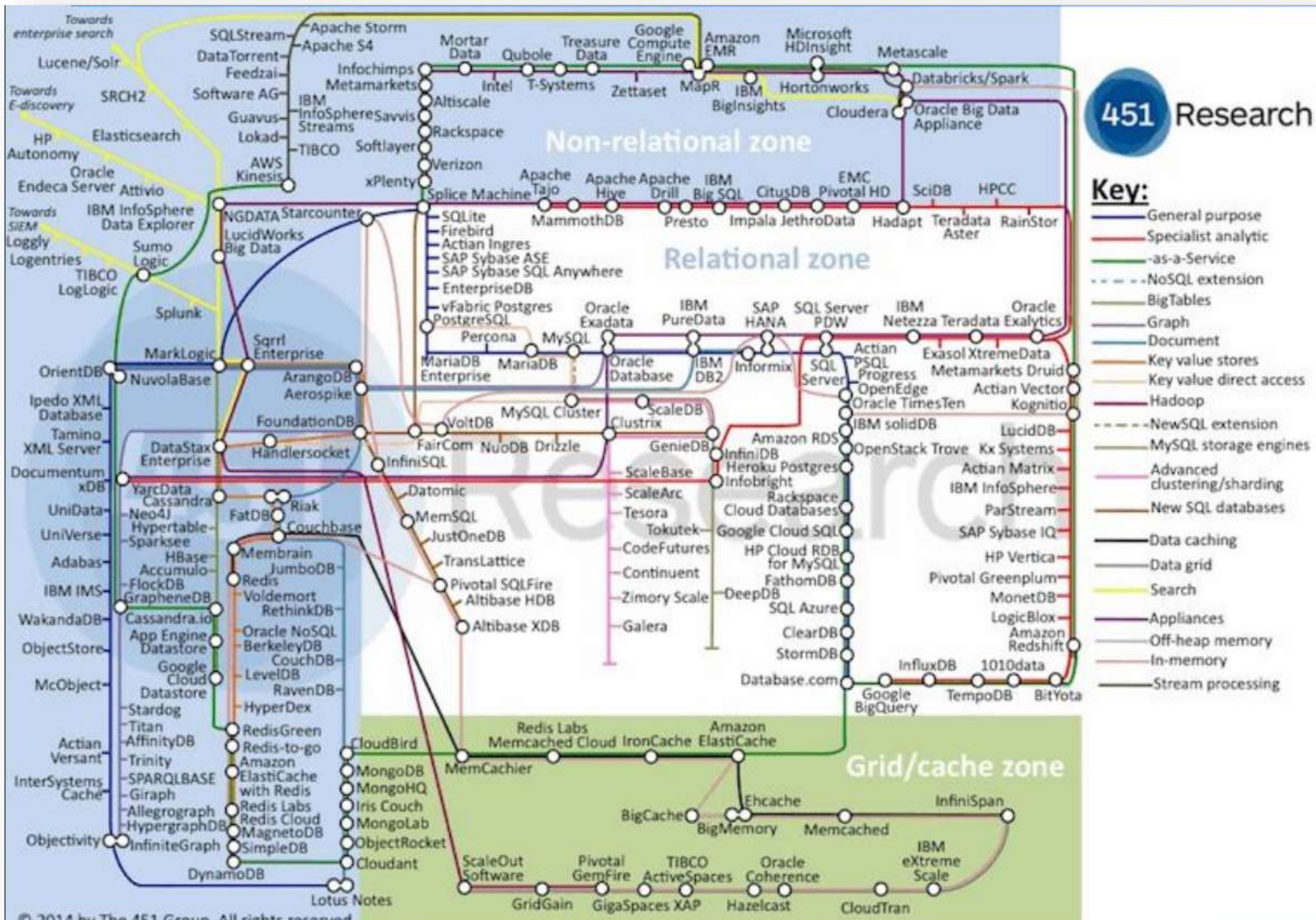
Cluster

Máquinas Virtuais

Data Lake

Cloud

Modelos e Arquiteturas



O reinado dos SGBDr está ameaçado?

- **Não... Veremos porque não!**
- **Os SGBDR tradicionais são maduros e oferecem:**
 - Eficiência, confiabilidade, desempenho e robustez.
 - Armazenamento e acesso seguros para grandes massas de dados.
 - Modelo relacional, álgebra consistente e linguagem SQL.
 - Garantias transacionais, concorrência, propriedades ACID.
 - Mão de obra abundante
- Porém.... nas últimas duas décadas, a comunidade e o mercado reconheceram que **nem todos os sistemas web e problemas de gestão e análises de Big Data** são bem resolvidos usando SGBDR tradicionais.

O reinado dos SGBDr está ameaçado?

* Há **duas razões principais** que motivam a TROCA de um SGBDr por NoSQL no atual mundo do “*Data Deluge e Big Data*”...

- 1) Necessidade de melhor desempenho
- 2) Necessidade de mais flexibilidade

- **Dados estão cada vez mais complexos em termos de...**

- Tamanho
- Estrutura
- Tipos
- Dispersão
- Taxas de Crescimento
- Proveniência
- Formas de Consulta, Armazenamento e Processamento

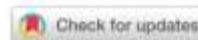


SQL databases v. NoSQL databases

Author:  [Michael Stonebraker](#) [Authors Info & Claims](#)

Communications of the ACM, Volume 53, Issue 4 • pp 10–11 • <https://doi.org/10.1145/1721654.1721659>

Published: 01 April 2010 [Publication History](#)



* Michael Stonebraker, criador do INGRES, H-Store, C-Store, SciDb, vencedor do Turing Award e professor do MIT...

NoSQL = “Not Only SQL”

- Desenvolvidos para se adequar aos **novos contextos da Web e suas novas tecnologias, infra-estruturas e dispositivos = Big Data = Século XXI**
- São conhecidos por ser um grupo de BD's distribuídos (boa execução em ambientes de clusters);
- Não tem esquemas pré-definidos (***schemaless***), não tem “relacionamentos” e “transações”, se baseiam em agregados de dados.
- **Linguagens de consultas menos poderosas em comparação com SQL.** ‘Substituição da “comunicação” via SQL por APIs simplifica o acesso aos dados.
- **Não seguem os princípios ACID dos sistemas relacionais, são mais flexíveis (CAP);**
- **Distribuição, Escalabilidade e Flexibilidade na programação são elementos centrais no NoSQL = persistência poliglota (Fowler, 2015).**
- **Consistência relaxada**
 - **Melhora a disponibilidade e desempenho no NoSQL;**
 - **Oferece menos garantias de consistência de dados comparado com BDr;**

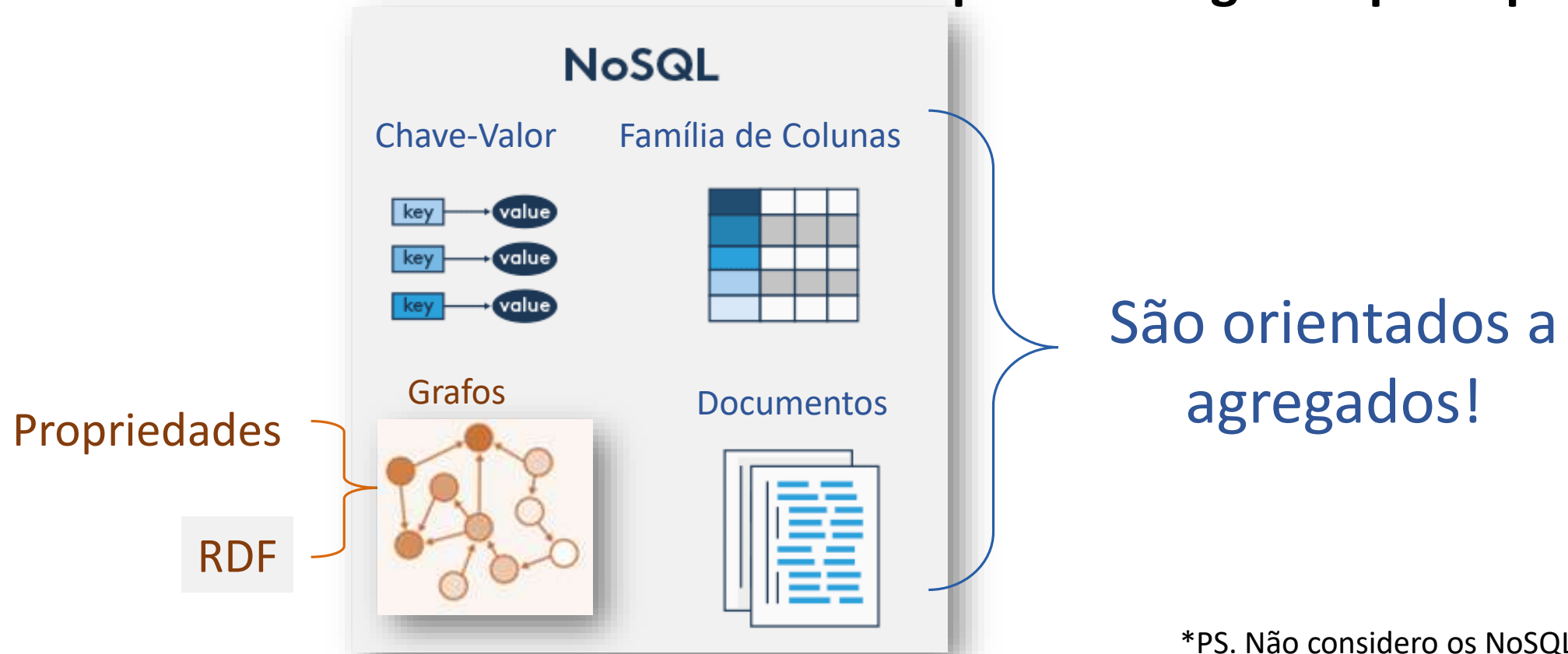
Características do Sistemas NoSQL (cont)

- **Escalabilidade Horizontal**
 - Sistema é expandido ao adicionar mais nós para armazenamento e processamento de dados à medida que o volume de dados aumenta
- **Maior disponibilidade e facilidade de replicação de dados e consistência eventual.**
- **Modelos de replicação**
 - Suporta replicação mestre-escravo e P2P.
- **Particionamento horizontal (*sharding*)**
 - Distribuição de carga nos acessos aos registros para vários nós.
- **Melhor desempenho para aplicações de Big Data**
- **Orientação a agregados de dados**

AULA 2

Sistemas NoSQL – Modelos de Dados

- Cada sistema NoSQL adota um modelo de dados diferente.
- Os modelos costumam ser divididos em **quatro categorias principais**



*PS. Não considero os NoSQL multimodelo

Sistemas NoSQL – Modelos de Dados

1 - **Orientados documentos**: Sistemas armazenam dados na forma de documentos (JSON). Documentos são acessíveis por meio de seu ID do documento, podem ser acessados rapidamente usando outros índices.

2 – **Orientados a chave-valor** : Sistemas possuem um modelo de dados simples, com base no acesso rápido pela chave ao valor associado; o valor pode ser um registro, um objeto, um documento ou até mesmo ter uma estrutura de dados mais complexa.

3 – **Orientados a coluna ou largura de coluna**: estes sistemas particionam uma tabela por coluna em famílias de colunas (particionamento vertical), em que cada família de colunas é armazenada com seus próprios arquivos.

4 - **Baseados em grafos**: os dados são representados como grafos e os nós relacionados podem ser encontrados percorrendo suas arestas por meio de expressões de caminho.

Outras Categorias

- Bancos de dados de objetos:
- Bancos de dados XML

Sistemas NoSQL – Agregados

- **Agregado é um conjunto de objetos relacionados que precisam ser tratados como uma única unidade de dados nos sistemas NoSQL**
 - É uma unidade de manipulação dos dados para facilitar o gerenciamento da consistência de dados
- **Em geral um agregado...**
 - Tem uma **estrutura mais complexa** que uma **tupla** do modelo relacional
 - Suas **modificações** são implementadas como uma **operação atômica** nos sistemas NoSQL
 - A **comunicação com os nós de armazenamento dos dados do NoSQL** é feita em termos de agregados.

Sistemas NoSQL – Agregados (cont)

- **Agregados facilitam o funcionamento dos sistemas NoSQL operando em clusters.**
 - **Exemplo:** É desejável minimizar o número de nós que precisam ser acessados para se responder uma consulta (menor latência e menos transferências de dados)
 - Os agregados sinalizam quais dados vão ser manipulados e que, portanto, deveriam ficar num mesmo nó do cluster.
- São a unidade natural para **replicação e particionamento de dados.**
- Agregados trazem **flexibilidade os trabalhos dos programadores**
 - **“incompatibilidade de impedância”** = Agregados têm uma estrutura lógica mais próxima dos objetos de dados em memória do que as tabelas de um BDr, facilita a manipulação de dados pelas aplicações.

Exemplo: Agregados no NoSQL X Relacional

“mundo relacional”



Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

BillingAddress		
Id	CustomerId	AddressId
55	1	77

OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Agregado de dados de “customer”

```
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      },
      {
        "id": 33,
        "orderPayment": [
          {
            "ccinfo": "1000-1000-1000-1000",
            "txnId": "abelif879rft",
            "billingAddress": {"city": "Chicago"}
          }
        ]
      }
    ]
  }
}
```

“mundo NoSQL”

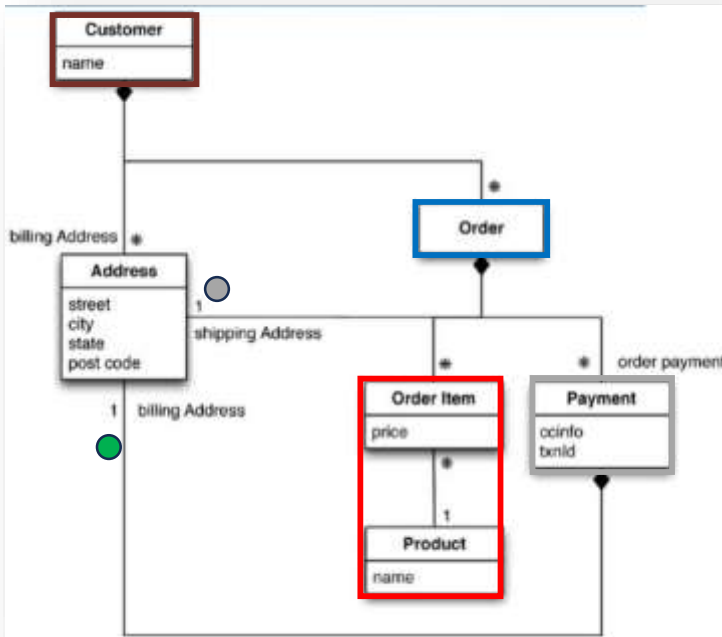


Diagrama de classes

Sistemas NoSQL – Problemas dos agregados

- Não existe o conceito de “**chave estrangeira**” para impor integridade referencial ou relacionamentos entre diversos agregados.
- Os sistemas NoSQL não conseguem usar a estrutura interna do agregado para tomar decisões relacionadas a **replicação e fragmentação dos dados**.
- Os sistemas NoSQL não conseguem manipular múltiplos agregados de **forma atômica** (não há transações ACID envolvendo mais de um agregado)
- O agregado não representa a organização lógica “intrínseca” aos dados; **reflete a forma como as aplicações acessam os dados**.
 - Quando é preciso acessar os dados de diferentes formas, o modelo relacional pode ser mais apropriada

Modelos de Distribuição de Dados no NoSQL

- **Dados** poder estar **fisicamente distribuídos** em **um ou mais nós**
 - **Servidor simples (nó único)** é o mais simples, **não há distribuição** de dados!
 - **Servidores (nós múltiplos)**, dados **precisam ser distribuídos**, isso requer orquestração e muito mais!
- A distribuição de dados pode ser implementada através de **duas estratégias ortogonais** em ambientes distribuídos.
 - **Fragmentação** (*sharding*) – Dados diferentes em nós diferentes.
 - **Replicação** – Os mesmos dados em múltiplos nós.
- **Fragmentação e Replicação podem ser combinadas para aumentar a resiliência do NoSQL** (Ex BDs de famílias de colunas)

Fragmentação de dados

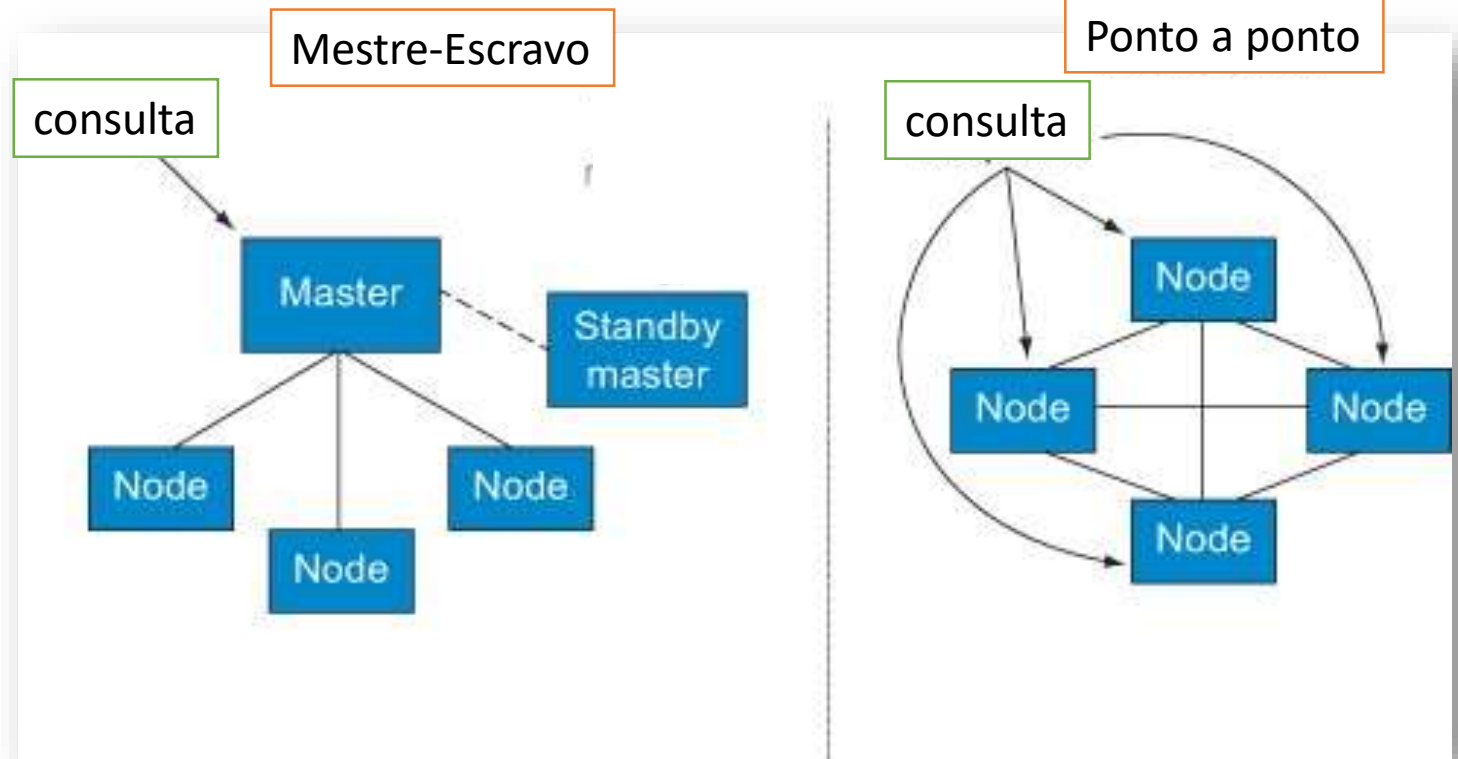
- **(RE)lembrando de aulas anteriores:** no “mundo dos BDDr” as relações podem ser fragmentada de modo horizontal, vertical e híbrida.
- **No mundo NoSQL, a fragmentação ocorre no nível dos agregados, cada nó tem seu dado.** Aumenta a escalabilidade horizontal.
- **Existem diversas formas de fragmentar agregados**
 - **Princípio da localidade geográfica de dados** - Dados ficam em nós onde são mais consumidos, reduz tráfego e atrasos na rede
 - **Princípio do equilíbrio de cargas no nós** - Dados ficam em nós distintos para equilibrar a sobrecarga dos nós
 - **Princípio da fragmentação lógica** – dados são logicamente fragmentados pelos aplicativos dentre os nós do cluster.

Fragmentação de dados

- Alguns NoSQL oferecem **autofragmentação nativa**. O próprio BD faz a alocação e gerencia os fragmentos através dos nós.
- **IMPORTANTE Fragmentação ...**
 - Só faz sentido com servidores múltiplos.
 - **Oferece escalabilidade horizontal** → amplia o desempenho das operações de leitura e gravação (reduz concorrência de acesso simultâneos aos nós em agregados múltiplos).
 - deve ser muito **bem planejada/implementada** → **agregará pouca resiliência se usada isoladamente** (se perder o nó, vai perder todos os dados!).

Replicação de Dados

- **(RE)lembrando de aulas anteriores** : no “mundo dos BDr” os bancos e relações também podem ser replicados em vários nós.
- No “mundo NoSQL”, a **replicação ocorre nos agregados**.
- A replicação de dados pode ser de dois tipos
 - Mestre-Escravo
 - Ponto a Ponto



Replicação de Dados: Mestre-Escravo

- Um nó é designado como **mestre** (primário/fonte de dados), responsável por atualizar os dados. Os demais são **nós escravos** (secundários) tem cópias dos dados. A replicação atualiza os escravos a partir do mestre.
 - **Nós escravos podem substituir o nó mestre em casos de falhas.**
- **Propagação de réplicas**
 - **Síncrona** → garante consistência de dados
 - **Assíncrona** → garante “eventual consistency”, não há a garantia de que o dado lido é o mais atual, mas há a garantia de que as atualizações serão propagadas para todos os nós em algum momento do tempo!
- **Útil para...**
 - Ampliar a **escalabilidade de leitura**, mas fica limitado a capacidade do mestre em reads, **não amplia a escalabilidade de write.**
 - Oferecer **resiliência de leituras** no caso de falha do mestre.
- **Riscos → Poderá ocorrer inconsistência de dados** em caso de falhas onde não houve propagação correta do mestre para os escravos!

Replicação de Dados: Ponto a ponto

- **Nesse tipo de replicação os mesmos dados são copiados para múltiplos nós aumentando sua disponibilidade.**
- Não tem os riscos do mestre-escravo, não há um mestre, todos os nós tem pesos iguais nas reads/writes.
 - **A perda de um nó não afeta o acesso ao armazenamento de dados.**
- **Útil para...**
 - Ampliar a **escalabilidade de leitura e escrita**,
 - Oferecer maior **resiliência nas op read/write** no caso de falha de nós.
- **Riscos → Inconsistência de dados** em caso atualização de um mesmo dados por pessoas distintas ao mesmo tempo!

Problemas de consistência de dados no NoSQL

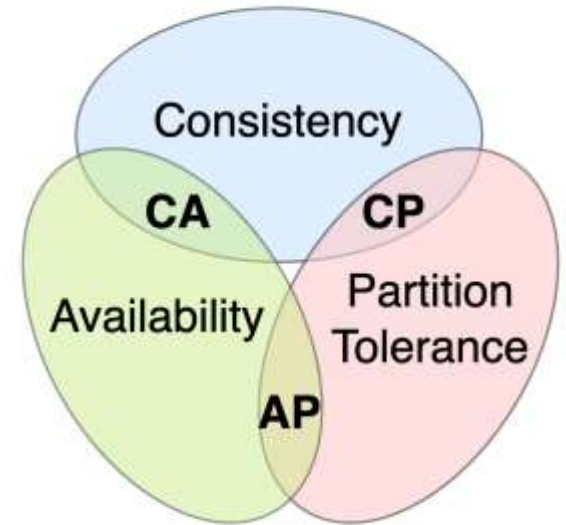
- Como não existem rígidos controles de transação no NoSQL, podemos ter **dois tipos de conflitos de consistência de atualização de dados**
- **Conflito Escrita-escrita** → quando dois clientes tentam escrever sobre os mesmos itens de dados (agregados) em um nó ao mesmo tempo.
- **Conflito Leitura-escrita** → quando um cliente lê dados inconsistentes de um nó meio da uma operação de escrita de outro cliente.
 - Sistemas distribuídos têm conflitos do tipo leitura-escrita porque alguns nós podem ter recebido uma atualização de dados enquanto outros nós ainda não foram atualizados (problemas: latência, erros de rede, sobrecarga de nós etc..)

Soluções de conflitos em Prol da consistência

- NoSQL pode adotar diferentes estratégias para resolução de conflitos.
- **Classicamente existem duas abordagens para evitar conflitos escrita-escrita:**
 - **Pessimista** →
 - 1) Bloqueia os agregados de dados para evitar que ocorram conflitos
 - 2) Grava as duas versões e deixa o usuário decidir qual versão do dado prefere usar
 - **Otimista** → Detecta os conflitos e depois tratam deles.
- Para obter **boa consistência de dados**, é preciso envolver **muitos nós** nas operações sobre os agregados de dados, mas **isso tem um custo!**
 - Pode aumentar a latência (tempo de resposta das operações)
 - As chances de falhas dos nós não podem ser negligenciadas!
- **Mecanismos empregados para solução de conflitos NoSQL:**
 - *Locks* [locais ou globais], no nível de agregado de dados
 - *Multiversion-Concurrency Control* (MVCC)

Teorema CAP

- Proposto por Brewer (2000) para sistemas distribuídos.
 - Um sistema distribuído é uma rede que armazena dados em vários nós ao mesmo tempo.
- CAP possui três propriedades:
 - **Consistência (C)** – capacidade de todos os clientes verem os mesmos dados em um cluster ao mesmo tempo.
 - **Disponibilidade (A)** – capacidade de acessar aos nós do cluster que não estejam em falha e atender as requisições dos clientes .
 - **Tolerância a partições (P)** – capacidade do cluster tolerar falhas de comunicação e ser capaz de dividir-se em várias partições (*split brain*) e permanecer atendendo as requisições dos clients
- **Somente duas podem ser atendidas simultaneamente!**
Alguns autores classificam os NoSQL como: CP, CA e AP
- **NoSQL não é ACID!**

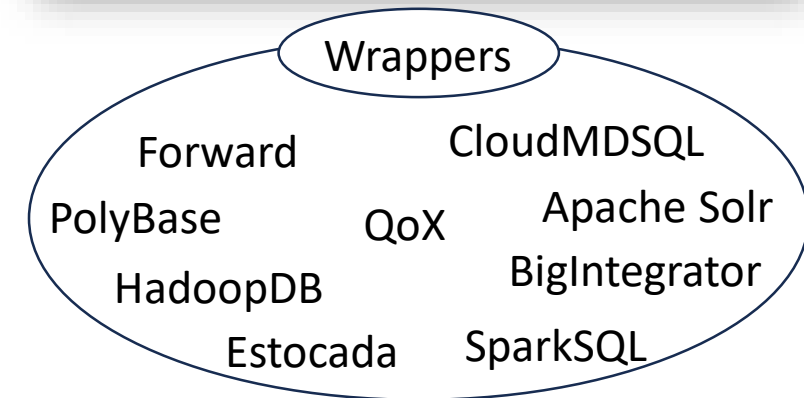
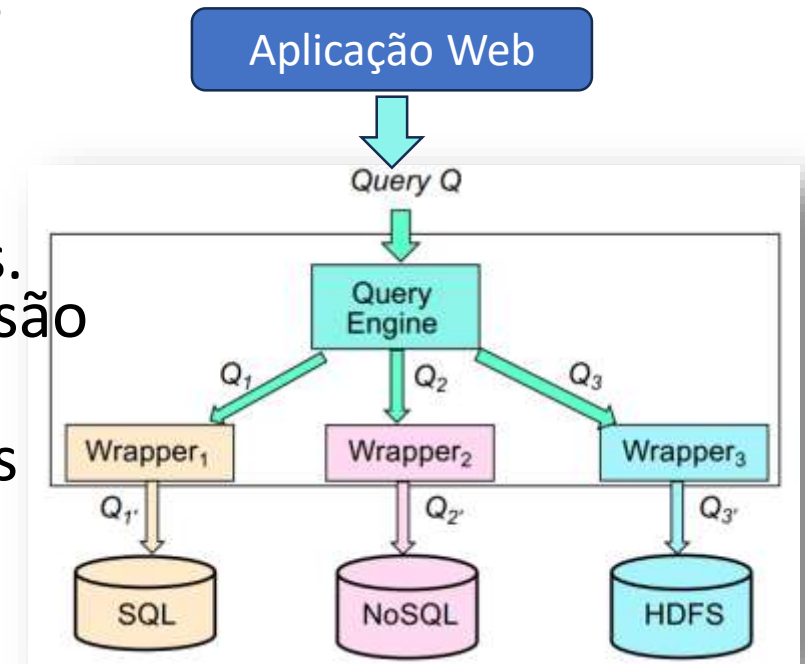


Propriedades BASE (opcional)

- Essencial escolher o tipo de NoSQL que entregue as características que a aplicação precisa.
- NoSQL fornece uma maneira flexível e fluida de manipular dados. Um novo conjunto de propriedades foi proposto (não há consenso!)
- **BASE (acrônimo para Basically Available, Soft State e Eventually Consistent)**
 - **Basically Available** – em vez de impor consistência imediata e rígida, NoSQL garante a disponibilidade dos dados via replicação nos nós (em algum momento) do cluster.
 - **Soft State** – devido à falta de consistência imediata, os dados podem mudar com o tempo (versionamento). BASE delega a verificação aos desenvolvedores.
 - **Eventually Consistent** – O fato não existir consistência imediata não significa que nunca se atingirá. Até que isso aconteça, as leituras de versões de dados é possível antes de atingir consistência;

Persistência Poliglota = Polystores (Stonebraker*)

- SGBDs são projetados para resolver diferentes problemas. Ex: Integração, ETL, movimentação e curadoria de dados são caros! **Não existe o “esperanto” nos SGBDs!**
- Há necessidades de acesso integrado aos dados em vários formatos em sistemas de armazenamento distintos
 - NoSQL, HDFS, SGBDr, SGBDD, Linked Data, etc...
- São classificadas de acordo com **grau de acoplamento: baixo, alto ou híbrido**
- **Wrappers, ótimo para integrar DE, DNE, DSE e Big Data.**
- **Importante área de pesquisa e desenvolvimento!**
 - Otimização de consultas, transações distribuídas, *load balance*, desempenho, interfaces, proveniência e muitas alegrias ;-)



* Stonebraker, M. (2015) The Case for Polystores. ACM blog - <https://wp.sigmod.org/?p=1629>

NoSQL – Família Chave-Valor

- **Modelo de dados é simples, baseado em pares <chave, valor-estruturado>**
- **Oferecem alto desempenho, disponibilidade e escalabilidade, armazenando dados em um sistema de armazenamento distribuídos.**
- **São fortemente orientados a agregados.**
 - Armazenam os **agregados** como um **conjunto de pares de chave-valor**, o id (**chave**) é um identificador único.
 - Os pares <chaves – valor> podem ser qualquer tipo de dado (desde objetos simples até objetos complexos)
- **Cada agregado tem uma chave**, usada na recuperação dos dados.
- **O agregado é “opaco” para o NoSQL, em geral:**
 - O valor do agregado é armazenado no BD como um BLOB.
 - O acesso ao agregado ocorre sempre por meio de buscas pela chave
 - Não há suporte para a definição de índices/chaves secundárias!

NoSQL – Família Chave-Valor (cont)

- **Principais Operações:**

- *Insert* (chave, valor)
- *Fetch* (chave) → busca/recuperação de valor
- *Update* (chave, valor)
- *Delete* (chave)

CHAVE	VALOR
16	título="NoSQL essencial"; ano="2014"; autor="Fowler"
Serra	altura="1,80"; nascimento="06-10-1970";
100	título="Barbie"; ano="2023"; gênero="fantasia"; estúdio="Warner"

- **As operações e as estruturas são extremamente simples**

- Alguns NoSQL permitem campos (não-uniformes) dentro do valor
- Alguns NoSQL permitem fazer *Fetch* sobre faixa de valores de chaves
 - Por exemplo, $16 < \text{chave} < 100$

NoSQL – Família Chave-Valor (cont)

- Esse tipo de NoSQL é:
 - **Altamente particionável** e permite **escalabilidade horizontal** em nível que outros NoSQL não alcançam.
 - Muito eficiente, o depósito de dados é uma “tabela hash” usada para acessar os pares <chave-valor>.
- **Exemplos de implementações**
 - Riak (<https://riak.com>)
 - Redis (<http://redis.io>)
 - Apache Cassandra (<https://cassandra.apache.org/>)
 - Amazon DynamoDB (<https://aws.amazon.com/dynamodb>)
 - Voldemort (<https://www.project-voldemort.com/voldemort>)

AULA 3

NoSQL – Orientado a Documentos

- **Modelo de dados são pares <chave, documento>**
- **Os documentos são estruturados em árvores hierárquicas autodescritivas**, podem constituídos por **coleções de documentos**, permite a definição de **documentos aninhados**
- **São fortemente orientados a agregados**
- **Cada agregado tem uma chave** usada na recuperação dos documentos.
- **Os documentos não são opacos para o NoSQL** (é possível ver a estrutura interna dos documentos), eles **podem ter estruturas distintas!**
- **NoSQL orientado a documentos é possível:**
 - Fazer buscas e criar índices sobre os campos do agregado.
 - Recuperar partes do documento (ao invés de recuperá-lo por inteiro)

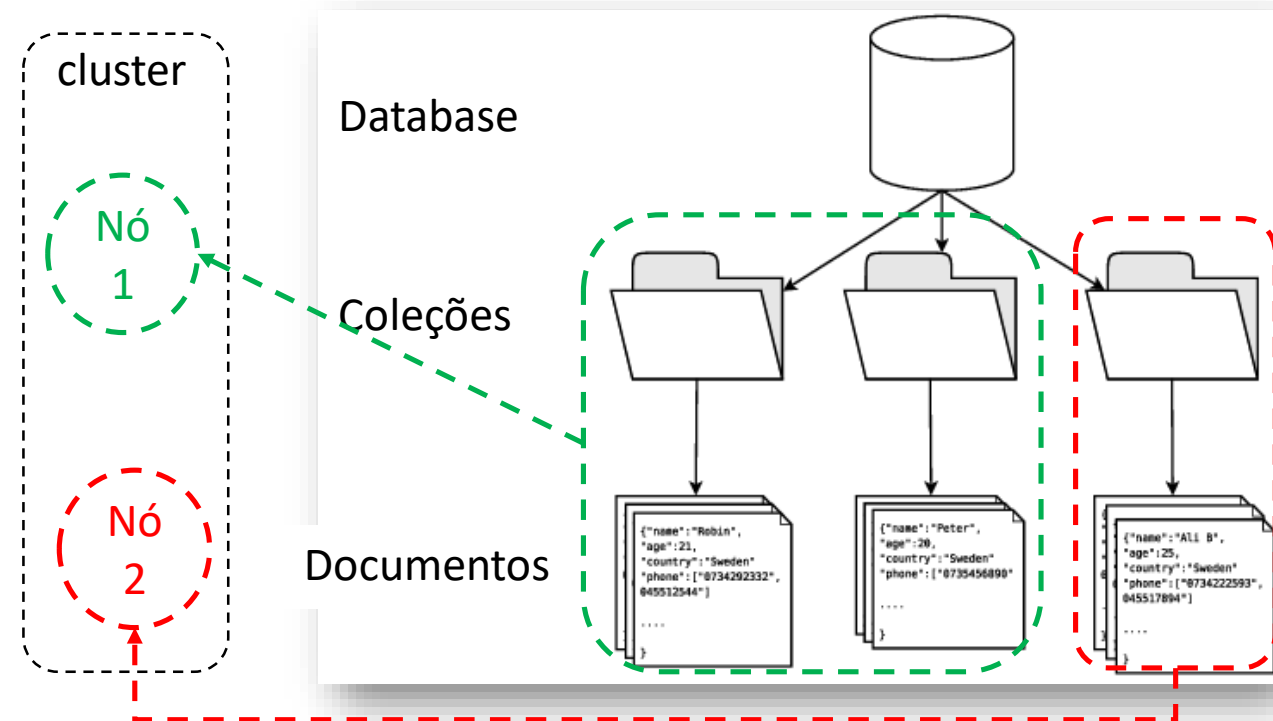
NoSQL – Família orientada a Documentos (cont)

Documentos são DSE (XML, JSON, BSON, etc.)

- **Operações básicas:**

- *Insert*(chave, documento)
- *Fetch*(chave) → Faz buscas em documentos pelos seus conteúdos
- *Update*(chave, documento)
- *Delete*(chave)

- **Não há uma linguagem padrão para as buscas**, varia conforme as implementações.



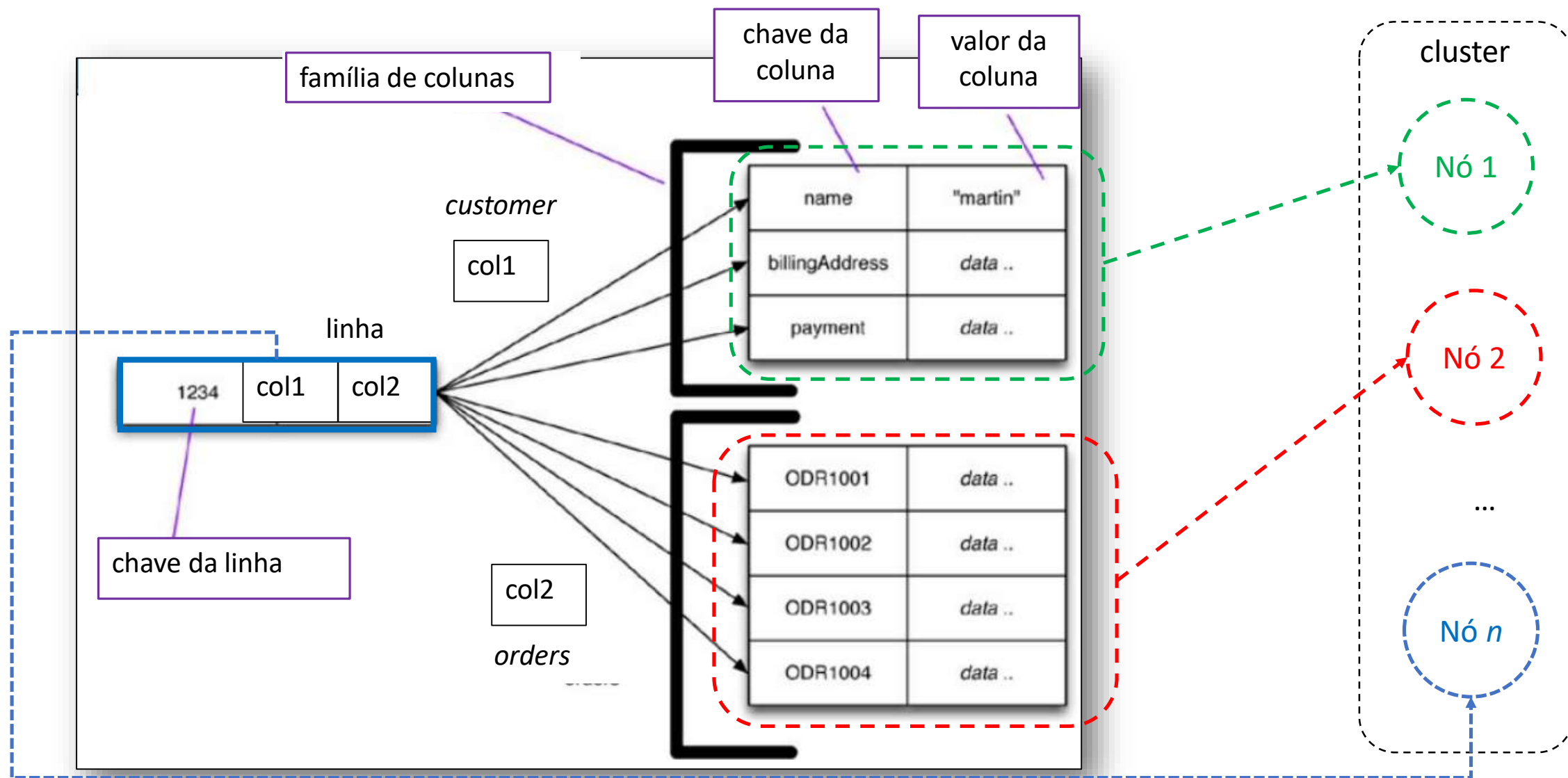
NoSQL – Família orientada a Documentos (cont)

- **Importante**
 - **Não implementa bloqueios explícitos** - **Consequência:** consistência ainda mais fraca que BDs chave-valor
 - Diferentemente dos chave-valor, em geral, **é possível definir índices secundários!**
- **Replicação e fragmentação**, são **altamente particionáveis** e permitem **escalabilidade horizontal** em nível que outros NoSQL não alcançam.
- **Exemplos de implementações**
 - MongoDB (<http://www.mongodb.org/>)
 - Couchbase (<http://www.couchbase.com/>)
 - CouchDB (<http://couchdb.apache.org/>)

NoSQL – Família de colunas

- **Modelo de dados adotado são pares (chave, família de colunas)**
- **BDs desse tipo armazenam os dados como linhas, onde cada linha possui uma chave e muitas colunas**
 - As linhas não precisam ter as mesmas colunas, novas colunas podem ser adicionadas a qualquer linha a qualquer momento (sem a necessidade de serem adicionadas às demais linhas)
 - Uma coluna pode ser uma “supercoluna”, ou seja, um contêiner para outras colunas.
- Uma **família de colunas é um agregado** de dados que geralmente são acessados juntos.

NoSQL – Exemplo de família de colunas



NoSQL – Família de colunas

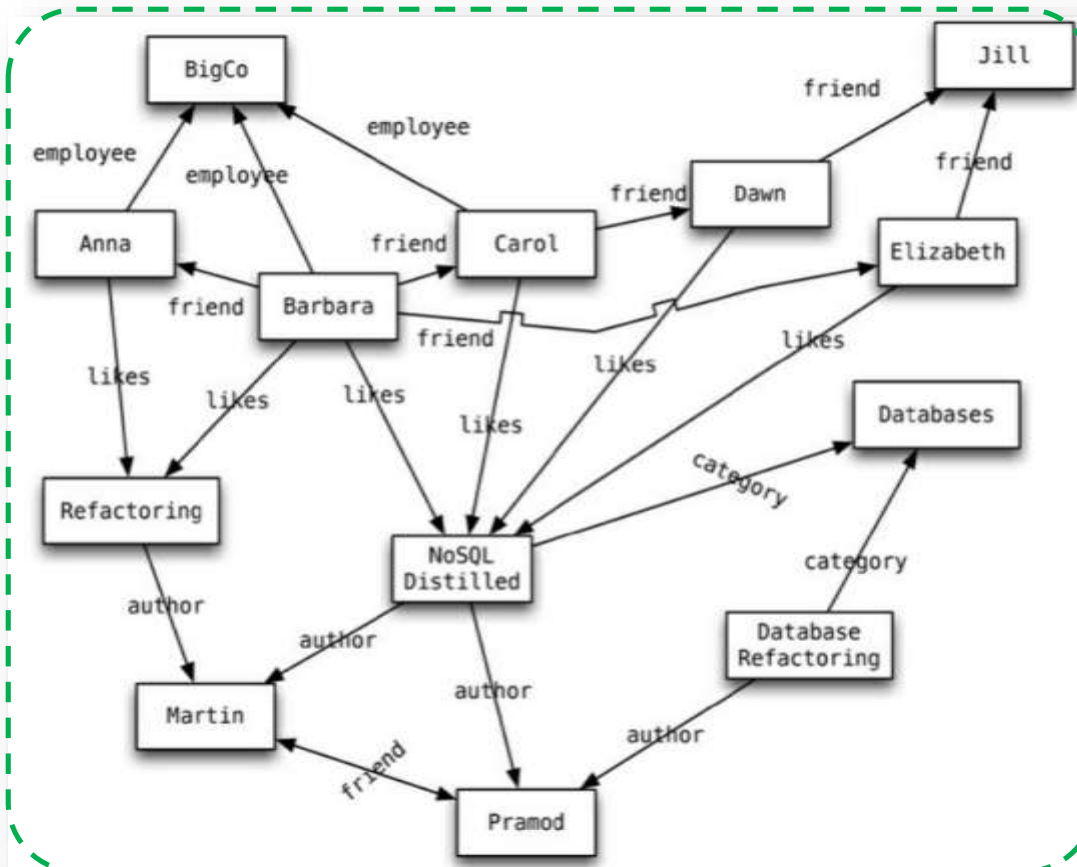
- Modelo oferece **altíssima escalabilidade**, possível dividir tanto linhas quanto colunas em diversos nós de um cluster.
 - As linhas podem ser divididas entre os nós **de acordo com o valor chave primária**.
 - A divisão ocorre por faixa de valores (e não por hash) → maior eficiência para buscas de linhas segundo faixas de chaves
 - As colunas são distribuídas entre os **nós de acordo com suas famílias** (pré-definidas pelo programador)
- **Suporta Replicação de dados e Fragmentação**
- **Exemplos de implementações**
 - Google Bigtable (<https://cloud.google.com/bigtable/>) – **Precursor da categoria!**
 - Usado em aplicações da Google. Usa GFS → Colossus
 - HBase (<https://hbase.apache.org/>) – Usa HDFS
 - MonetDB (<https://www.monetdb.org/>)

NoSQL – Grafos

- **Modelos de dados baseado em vértices (ou nós) (entidades) e arestas (relacionamentos) e pares chave-valor**
 - Nós podem ter propriedades (incluindo ID)
 - Arcos podem ter rótulos ou papéis
- **Grafos são DAGs** (estrutura de dados de propósito geral adequados para qualquer dado conectado)
 - São ótimos para grandes volume e densidades de dados,
- **Grafos podem ser classificados como:**
 - **Orientados a Propriedades** → Big Data / Data Science é uma tendência
 - **Orientados a semântica (RDF)** → Web Semântica é a tendência → Grafos do conhecimento! Prox. aula
- **Diferente dos demais NoSQL, BDs de grafos...**
 - Não são orientados a agregados!
 - Suportam processamento transacional (implementa as propriedades ACID!)
 - Fragmentação de dados é um problema, dados devem ser mantidos em uma único nó.

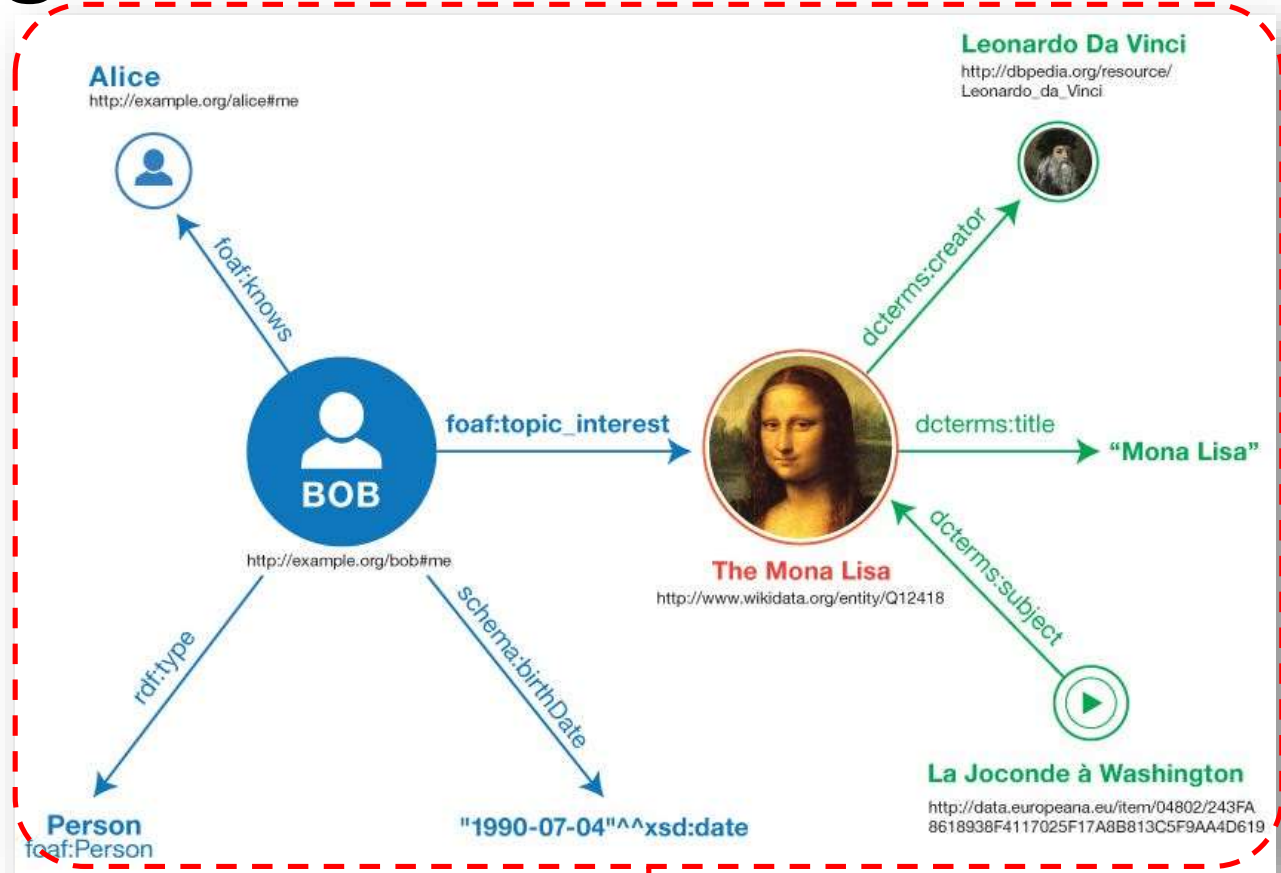
São “inspirados” em Euler, Codd, teoria dos grafos

NoSQL – Exemplos de grafos



grafo de
propriedades

Nó 1



grafo RDF
(s, p, o)

LOD

NoSQL – Grafos (cont)

- Os nós podem ter diferentes tipos de relacionamentos (**dados complexos**); não há restrições quanto à quantidade de relacionamentos que os nós podem ter
- **Não há uma linguagem padrão para as consultas**, varia conforme as implementações NoSQL.
- **As consultas são muito velozes e podem envolver:**
 - Passo único com expressões de caminho ou recursões completas
 - Condições sobre propriedades e rótulos
- **Percorrer os relacionamentos é uma operação muito rápida** (não precisam ser calculados no momento da consulta; eles já estão persistidos)

NoSQL – Grafos (cont)

- **NoSQL grafos não é uma ideia nova**, mas muito atual para o momento do *Data Deluge* e *Big Data*.
- **Vários exemplos de implementações**
 - **Grafos de propriedades**
 - Neo4J (<http://www.neo4j.org/>)
 - Linguagem de consultas em Cyphe (“perl for graphs”)
 - Giraph (<http://giraph.apache.org/>)
 - **Grafos de RDF**
 - Virtuoso triplestore (<https://virtuoso.openlinksw.com/>)
 - GraphDB (<https://graphdb.ontotext.com/>)
 - Linguagem de consultas em SPARQL (“SQL for linked data”)

Quadro comparativo principais NoSQL

Vendor	Product	Category	Comments
Amazon	DynamoDB	KV	Proprietary
Apache	Cassandra Accumulo	KV Tabular	Open source, Orig. Facebook Open source, Orig. NSA
Couchbase	Couchbase	KV, document	Origin: MemBase
Google	Bigtable	Tabular	Proprietary, patents
FaceBook	RocksDB	KV	Open source
Hadoop	Hbase	Tabular	Open source, Orig. Yahoo
LinkedIn	Voldemort Expresso	KV Document	Open source ACID transactions
10gen	MongoDB	Document	Open source
Oracle	NoSQL	KV	Based on BerkeleyDB
OrientDB	OrientDB	Graph, KV, document	Open source, ACID transactions
Neo4J.org	Neo4J	Graph	Open source, ACID transactions
Ubuntu	CouchDB	Document	Open source

Modelo de Programação Map Reduce

- Map-Reduce é um **modelo de programação** para **processamento** de **grandes conjuntos de dados em ambientes distribuídos** (Dean, Ghemawat-Google, 2004)
 - Base teórica é bem simples, cada ação é especificada em termos de um **pipeline** de tarefas com as funções de **mapeamento** e **redução** de dados.
 - As tarefas rodam paralelamente nos nós de cluster, armazenamento é fornecido por DFS.

Uma visão conceitual do **Map-Reduce** consiste em três passos:

1. A função **Map** transforma cada item do input em zero ou mais **pares chave-valor**. (*Leitura sequencial dos dados*).
 2. Agrupar e ordenar todos os **pares chave-valor** em **coleções com chaves idênticas**. (*Resultado intermediário*)
 3. A função **Reduce** produz valores de saída para a **chave correspondente** para cada **coleção de valores agrupados**. (*Escrita dos resultados finais*)
- **Map-Reduce é usado em...**
 - Análise de links, Processamento de grafos, Aprendizado de Máquina e muito mais

Funções Map e Reduce

• Map

- **Entrada** : Um ou mais fragmentos arquivos do DFS
 - Transforma cada fragmento de arquivo em um conjunto de pares <chave, valor>, através da **função de mapeamento definida pelo usuário**.
- **Saída**: conjunto de pares <chave, valor>

• Reduce

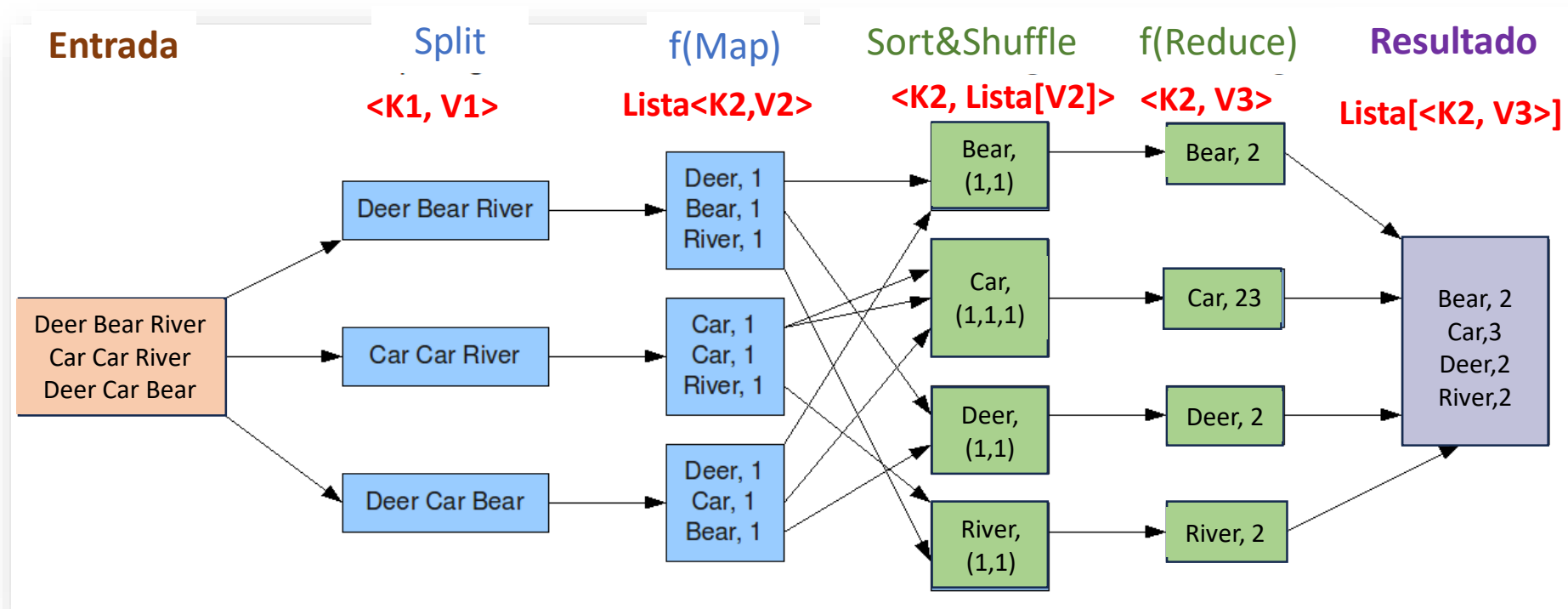
- **Entrada** : Um conjunto de pares <chave, [lista de valores]>
 - Combina os valores associados com uma mesma chave, através da **função de redução definida pelo usuário**.
- **Saída**: Um ou mais fragmentos de arquivos no DFS contendo pares <chave, valor>

```
map(key, value):  
// key: nome do arquivo;  
// value: conteúdo do arquivo  
for each word w in value:  
    emit(w, 1)
```

```
reduce(key, values):  
// key: uma palavra;  
// values: contador  
result = 0  
for each count v in values:  
    result += v  
emit(key, result)
```

Exemplo: Map-Reduce contagem de palavras

- Imagine que uma **coleção** de arquivos... O input pode ser logs de web site, livros ou **qualquer outra coisa**.
 - Como contar as ocorrências das palavras em cada arquivo?



Atividade Extra-Classe

- **Hands on** “Neo4j Fundamentals” no <https://graphacademy.neo4j.com/courses/neo4j-fundamentals> (1 hora)



- **Data de Entrega:** Próxima aula, envio das respostas do exercício do **Hands on** por email.
- **Método Avaliação:** lista individual.
- **Pontuação:** Vale até 1.0 na P1

Leituras Complementares

- ARTIGO. Stonebraker, M. (2015) “The Case for Polystores”. ACM blog - <https://wp.sigmod.org/?p=1629>
- PALESTRA apresentada no SBBD (2020) por Valduriez, P. “Principles of Distributed Database Systems: spotlight on NewSQL”,
 - Ótimo resumo de NoSQL e introdução ao NewSQL!

Data Vault | @Datavault UK | www.data-vault.com | Data Vault Technology Landscape | Winter 2018/2019

Reporting Systems

- Dashboards and Reporting Engines:** Qlik, Google Data Studio, SAS, TIBCO Spotfire, Pentaho, Oracle Hyperion, Microsoft SQL Server, Tableau, Pivotal Greenplum, Yellowfin, Cognos, Birst, Power BI, Information Builders, JasperSoft, Chartio, QuickSight, Oracle Business Intelligence, SAP Business Objects, Informatica.
- Data Science (including AI and ML):** KNIME, SciPy, SPSS, Mahout, Python, MATLAB, SAS, Jupyter, R, TensorFlow, Databricks, GNU Octave.
- Streaming:** Quibole, Apache Kafka, Amazon Kinesis, Nifi, Samza, Alookma, StreamSets, RabbitMQ, TIBCO Spotfire, Elasticsearch, Informatica, Apache Flink, Apache Storm.

Integration

- ETL/ELT:** Microsoft SQL Server, Tamr, Alouma, Fivetran, Alteryx, Talend, Matillion, StreamSets, DiYOTTA, Ab Initio, IBM DataStage, CloverDX, Bimi, BimlFlex, WhereScape, Pentaho, DataBlade, Informatica.
- Cleaning:** Waterline Data, WS, IBM, Alteryx, Tamr, IRI.
- Scheduler:** Talend, WS, Alteryx, BimlFlex, Cron, Oozie.

Database Systems

- Databases/File systems:** Redshift, Amazon DynamoDB, Cassandra, Google BigQuery, CouchDB, Amazon Aurora, MySQL, MongoDB, Teradata, MariaDB, Snowflake, Oracle Database, Hadoop HDFS, IBM DB2, SAP S/4 HANA, InfoBright, PostgreSQL, Microsoft SQL Server.
- Data Formats:** XHTML, XML, JSON, CSV, COBOL, ORC, Avro.
- SQL Engines:** Presto, Hive, Cloudera Impala, Spark.

Platform and DevOps

- Platform:** Amazon Web Services, Windows Azure, Google Cloud Platform, Rackspace.
- DevOps:** Jenkins, Ansible, Planview LeanKit, Terraform, Jira Software, CollabNet VersionOne.
- Platform Utilities:** Apache Knox, Dynatrace, Apache Ranger, Apache Falcon, Ganglia.
- Version Control:** CVS, Git, Mercurial, Subversion.
- Backend Languages:** Scala, Java, T-SQL, Python, Oracle PL SQL.

©Datavault is a trading name of Business Thinking Ltd. All Trademarks acknowledged. While we strive to make this information as accurate as possible, we make no claims, promises, or guarantees about the accuracy, completeness, or adequacy of the contents, and expressly disclaims liability for errors and omissions.

Extra: Comparativo NoSQL x SQL

Característica	SQL	NoSQL
Modelo relacional (DE)	Sim	Não
Suporta schema dinâmico	Não	Sim
Propriedades ACID	Sim	Não, Teorema CAP (BASE)
SQL	Sim	Não
Escalabilidade	Vertical	Horizontal
Distribuição	Sim (BDDr)	Sim
Suporta consultas complexas	Sim	Sim, Não tão bom
OLTP	Sim	Não tão bom
OLAP	Sim	Sim

Extra: SQL x NoSQL X NewSQL

Feature	SQL	NoSQL	NewSQL
Relational Property	Yes	No	Yes
ACID	Yes	No, support CAP.	Yes
SQL	Yes	No	Yes
OLTP	Inefficient for OLTP databases.	Supports but it is not the best suited.	Fully supported.
Scaling	Vertical scaling	Horizontal scaling	Vertical + Horizontal scaling
Distributed Databases	No	Yes	Yes

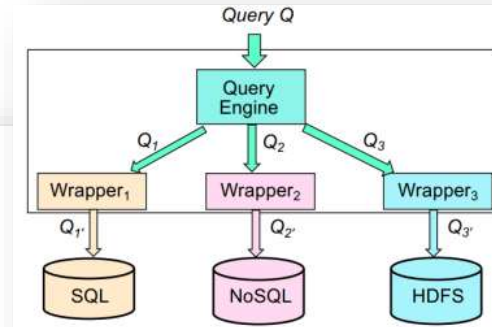
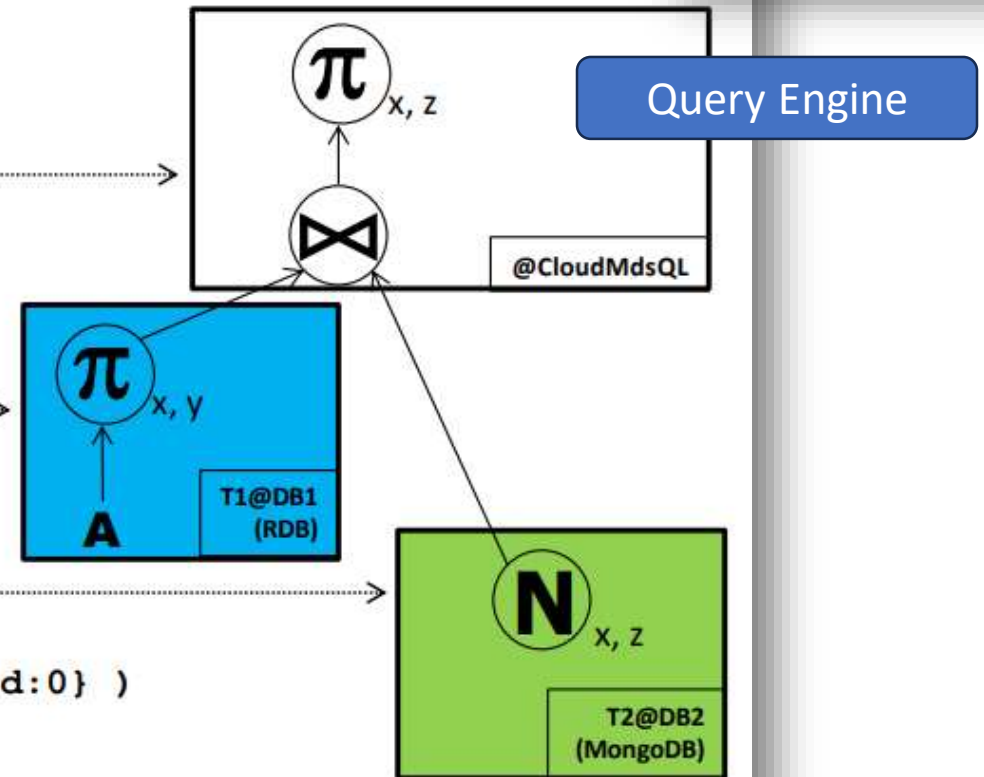
Exemplo de subconsultas em Polystores

- A query in CloudMdsQL* that integrates data from
 - DB1 – relational (RDB)
 - DB2 – document (MongoDB)
 - Expressed as a Native subquery

```
/* Integration */
SELECT T1.x, T2.z
FROM T1 JOIN T2
ON T1.x = T2.x
```

```
/* SQL sub-query */
T1(x int, y int)@DB1 =
( SELECT x, y FROM A )
```

```
/* Native sub-query */
T2(x int, z string)@DB2 = {*
  db.B.find( {$lt: {x, 10}}, {x:1, z:1, _id:0} )
*}
```



Extra: Quadro comparativo de Polystores

Polystore	Objective	Data model	Query language	Data stores
Loosely-coupled				
BigIntegrator	Querying relational and cloud data	Relational	SQL-like	BigTable, RDBMS
Forward	Unifying relational and NoSQL	JSON-based	SQL++	RDBMS, NoSQL
QoX	Analytic data flows	Graph	XML based	RDBMS, ETL
Tightly-coupled				
Polybase	Querying Hadoop from RDBMS	Relational	SQL	HDFS, RDBMS
HadoopDB	Querying RDBMS from Hadoop	Relational	SQL-live (HiveQL)	HDFS, RDBMS
Estocada	Self-tuning	No common model	Native QL	RDBMS, NoSQL
Hybrid				
SparkSQL	SQL on top of Spark	Nested	SQL-like	HDFS, RDBMS
BigDAWG	Unifying relational and NoSQL	No common model	Island query languages	RDBMS, NoSQL, Array DBMS, DSMSs
CloudMdsQL	Querying relational and NoSQL	JSON-based	SQL-like with native subqueries	RDBMS, NoSQL, HDFS